Teaching Coding Agents to Master *Spreadsheets*

Nuno Campos

AI Engineering London — April 2026

Duration: 20 minutes. Slides + code examples.

50% → 92%

— 4 months, multiple architectures, and many dead ends

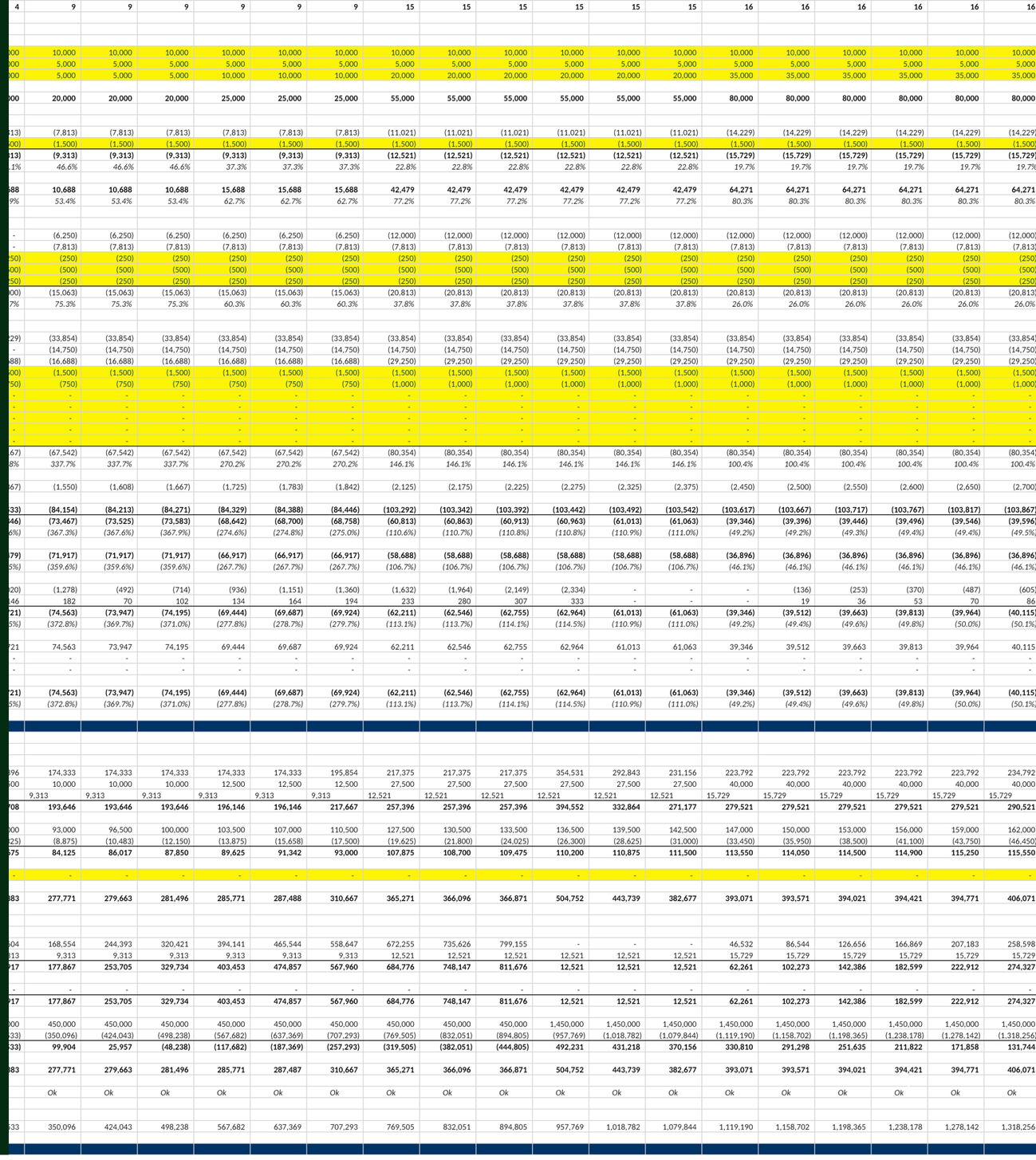
— What mattered most: replacing 15 discrete tools with one REPL

This is about building an agent that reads and modifies Excel spreadsheets. We started at 50% accuracy on a financial analysis benchmark and got to 92%. The talk covers what actually moved the needle and what didn't. The short version: we kept building more tools, and the agent kept wanting to program. Once we let it, the results followed.

The problem

A human sees: revenue table, assumptions, P&L summary

An LLM sees: 10,000 cell values, `=SUMPRODUCT((B$3:B$50="NEW")*(G$3:G$50))`, formatting metadata



Spreadsheets are deceptively hard for AI. A human glances at a financial model and instantly sees structure -- there's a revenue table here, assumptions in that yellow corner, a chart summarizing the P&L. They know "Q4" means the fourth column, parentheses mean negative numbers, the cell labeled "EBITDA" is derived from the ones above it.

An LLM sees none of this. Give it a 20-sheet financial model and ask "what's the IRR?" — it knows the concept, but it has to figure out which of 10,000 cells contains the answer, which sheet it's on, and whether that number is an input or a calculated output.

Attempt 1: Turn the spreadsheet into a database

— Excel → SQLite (every cell, formula, merged region, color block)

— Agent: LangGraph + GPT-5, tools for SQL query / write cell / insert row

Result: 50% — then 73% after fixing a one-character extraction bug

Our first instinct: transform the problem. LLMs are good at SQL, so we built a pipeline that converts entire workbooks into SQLite databases. The agent had a handful of tools -- run a SQL query, write to a cell, insert a row. Each tool call was a separate round-trip to the model.

Tested against SpreadsheetBench, 130 tasks. First result: 50%. Then a one-character bug in the extraction code -- a wrong function argument -- turned out to be corrupting data for a large fraction of tasks. Fixing it moved us to 73%. The agent had looked confused, so we blamed the model. It was reasoning correctly over bad inputs.

Three specialized agents:

1. Block discovery (low effort) — identifies workbook structure
2. Edit agent (high effort) — 5-step process: disambiguate, define end state, plan, execute, verify
3. Question agent (read-only) — answers questions

Key finding: "Define the end state before you touch a cell" was the most impactful prompt instruction

Next we split into three agents. The important one was the edit agent's five-step reasoning process: disambiguate the request, define the desired end state, plan the implementation, execute, verify. This changed the kind of errors we got. Without it, the agent made irreversible mistakes mid-execution. With it, most errors surfaced during planning, where they're cheap. This worked better than giving the agent better tools, more context, or a stronger model.

But the architecture was rigid. Discovery ran once upfront and couldn't be revisited. The three-agent pipeline meant context couldn't flow between stages. And we still had 15+ tools the agent was juggling.

The dead ends

Representation	Why it failed
TSV views	Lost formatting and structural context
SQL views	Flat tables didn't capture visual structure
HTML tables	Too verbose, consumed too many tokens
DOT graphs	Useful for analysis, not for interaction
XML/XSLT	LLM struggled with the syntax

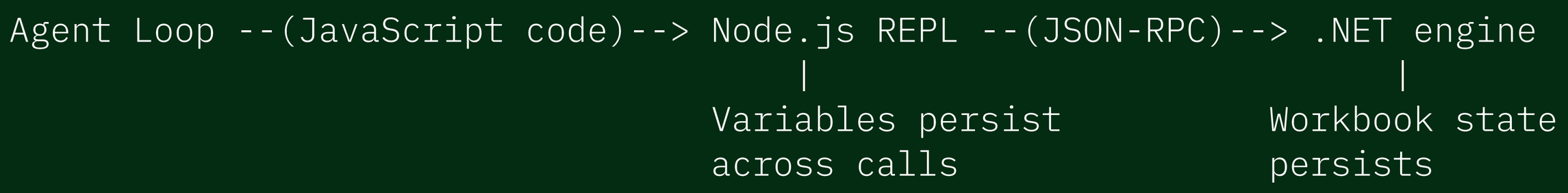
None of them worked as a general-purpose representation — but two informed what came next.

We pivoted to TypeScript and .NET for better Excel fidelity, then spent two weeks trying every possible way to represent a workbook to the LLM. None worked as a standalone representation, but two turned out to be useful as methods inside the REPL.

TSV is compact and communicates surrounding context in few tokens. It works especially well when each cell includes its address — off-by-one counting mistakes are common otherwise — and when formulas are included alongside values. This became `readRangeTsv`, one of the most-used API operations.

HTML was a step in the right direction for when layout, formatting, and whitespace matter — but only when rendered to a raster image. We ended up writing a rendering engine that previews any range to PNG. That became the visual verification step in the feedback loop.

We replaced all 15 tools with a persistent Node.js REPL and a spreadsheet API.



On November 23rd, we replaced all 15 tools with one: a persistent Node.js REPL with access to a spreadsheet API -- about 50 operations for reading, searching, tracing formulas, and writing. Instead of "read this cell", "search for this label", "write this value" as separate tools, the agent writes JavaScript. Variables persist across calls. 90-second timeout per execution, 5MB output limit. The REPL runs in a sandboxed Node.js worker with the permission model enabled -- read-only filesystem access to the workspace, no writes, no network, no child processes.

Before (10-15 tool calls):

```
Tool call 1: list_sheets()           -> ["Summary", "Data", "Inputs"]
Tool call 2: read_range("Summary!A1:D10") -> cell values
Tool call 3: find_cells("Revenue")   -> [matches]
Tool call 4: read_cell("Summary!C10") -> "$1,234,567"
...
```

After (1 tool call):

```
const sheets = await xlsx.listSheets(wb);
const summary = await xlsx.readRangeTsv(wb, `${sheets[0].name}!A1:D10`);
const revenue = await xlsx.findCells(wb, "Revenue", { context: 1 });
console.log(sheets, summary, revenue);
```


Before: exploring a workbook took 10 to 15 tool calls. Each one is an LLM round-trip -- the model decides what to do, formats the tool call, waits for a result, interprets it, decides the next step.

After: the same exploration in one call. Three operations, all results visible together. But it goes further than just batching.

Code mode vs. REPL

Code mode — the agent writes scripts instead of making tool calls. Operations compose naturally. 50-line scripts are common.

REPL = code mode + persistent state — variables survive across calls. The agent writes shorter scripts, reasons between them, builds understanding incrementally.

```
// Code mode: one long script, print everything at the end
// REPL: explore, reason, continue
const revenue = await xlsx.findCells(wb, "Revenue", { context: 1 });
console.log(revenue);
// → agent reasons about the output, then writes the next script
```

Result: small accuracy gain on harder tasks — the agent can course-correct mid-exploration.

Code mode is already a big improvement over discrete tools. The agent writes a complete script -- find the sheets, read a range, search for a label, print the results -- all in one call. 50+ lines is common. But everything has to be planned upfront.

Persistent state changes the agent's behavior. It writes shorter scripts, printing fewer items each time, and interleaves reasoning with execution. It can look at output, think about what to explore next, and continue from where it left off. On harder tasks -- multi-sheet analysis, ambiguous labels -- this led to a consistent accuracy improvement. There's also a latency gain: the workbook stays open across calls, so the agent isn't paying the cost of reopening and reparsing the file on every invocation.

Two more reasons the REPL worked: flexible exploration -- conditionals, loops, error recovery within a single call, which discrete tools can't do. And API evolution -- adding traceToInputs and traceToOutputs from months of Rust formula work was just adding two functions. No tool schema changes, no registration. The entire API surface ships as a single skill prompt, recently compressed from 61k to 34k characters to fit in agent context windows.

The results

Date	Pass rate	Tasks
Nov 30	**74%**	104
Dec 11	**88%**	167
Dec 14	**92.1%**	165

Zero timeouts. 50-second average runtime.

18 points in two weeks from compounding small gains:
better search, new API functions, improved docs, backend bug fixes

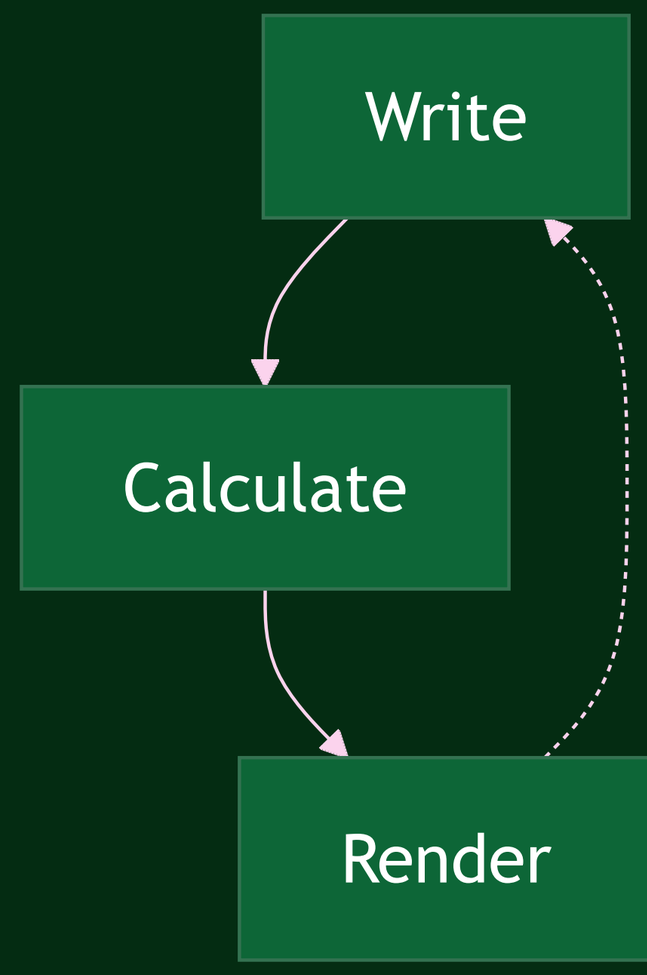
No single change after the REPL was dramatic on its own. Better fuzzy search, formula tracing functions, improved system prompt documentation, bug fixes in the .NET backend. But each one removed a class of failures, and the effects compounded -- 18 points in two weeks.

The zero-timeout number matters. Before the REPL, timeouts were a significant failure mode. After it, every task completed within budget.

The verification loop

The formula engine and renderer close a loop the agent uses to check its own work.

This held across three successive frontier model releases. Each new model used the same loop more effectively.



The .NET formula engine and visual renderer close a feedback loop. The agent writes to a cell, the engine recalculates all dependents, the agent checks for formula errors, and if needed renders a region to verify the result visually. This matters because the formula engine is the source of truth -- early on our agent kept calculating things in JavaScript instead of reading from the engine, and its arithmetic was sometimes wrong. The verification loop makes it natural to use the engine: write, recalculate, read the result back.

We tested across Opus 4.6, GPT 5.4, and Gemini 3.1 Pro on a 256-task financial QnA dataset. Compared to the same models with no witan tools or skills, the advantage was consistent: +8 percentage points for Codex/GPT (77.9% vs 69.9%), +5 for Claude Code (72.5% vs 67.2%). Latency was lower too -- 41s vs 49s for Codex, 46s vs 55s for Claude Code. The engines compound with model capability rather than being replaced by it.

This requires a real formula engine. openpyxl stores formulas as strings and can't recalculate. The standard workaround is shelling out to LibreOffice, which doesn't support LAMBDA or array formulas, corrupts OOXML on round-trip, and fails silently in containers. In our benchmark, 9 out of 10 tasks where the agent fell back to openpyxl as its primary tool failed. An incomplete engine as feedback makes output worse -- it gives the agent incorrect intermediate results to reason over.

The **REPL** is an interface — the best one today, because coding is where models are strongest.

The **engines** — formula calculation, rendering, linting — are the durable part. They're what close the verification loop, and they compound with each new model.

If agents become as capable at computer use as they are at coding, the interface might change. The engines won't.

The REPL works today because coding is the dominant model capability. But the jagged frontier of capability keeps moving. If computer use catches up — if agents can interact with a spreadsheet visually as effectively as they can write code against an API — then the best interface might look very different.

What won't change is the need for high-fidelity formula calculation, rendering, and linting. These are what let the agent verify its own work. They compound with model capability rather than being replaced by it. Three successive model releases confirmed this — each new model used the same verification loop more effectively.

We'd built a separate set of CLI commands — `find`, `calc`, `render`, `lint` — designed as standalone tools any agent could call.

We tested them against plain openpyxl on 20 QnA tasks. Same model, same runner.

We expected ours to win. It had a formula engine, semantic linting, visual rendering. openpyxl can't even recalculate formulas.

By February, we had two Witan products. The REPL that hit 92% on our internal agent. And a set of standalone CLI commands -- find, calc, render, lint -- designed for external use. We wanted to know if these CLI tools could beat the default. So we tested them against plain openpyxl on 20 QnA tasks.

openpyxl won

	openpyxl	Witan CLI
Pass rate	**85%**	**70%**
Avg tool calls	21	42

The default approach won by 15 points.

85 to 70, with half the tool calls.

Why it lost

Every CLI command was a separate process — startup, shell parsing, .NET engine initialization. That overhead compounded:

1. **A recalculation bug** made each command take 50-130 seconds. 20+ commands per task meant timeouts.
2. **Shell escaping** across three layers (SDK, zsh, .NET parser) broke on sheet names with spaces.
3. **Our documentation** had the quoting examples backwards. The agent followed our wrong instructions faithfully.

openpyxl is an in-process library. No subprocesses, no shell, no infrastructure to go wrong.

The root cause was structural: each CLI command spawned a process, parsed arguments through a shell, and initialized the .NET engine. That's three layers of overhead and three layers of potential failure per operation. With 20+ operations per task, the overhead dominated.

The specific bugs -- per-cell recalculation instead of batch, shell escaping across three quoting layers, backwards documentation -- were symptoms. Any one could have been fixed, but the architecture meant new failure modes would keep appearing. openpyxl avoided all of this by being a library call.

There were other issues too: intermittent empty API responses, over-reliance on PNG rendering for data extraction, and no automatic .xls-to-.xlsx conversion.

This test showed why the REPL worked where the CLI didn't: **a single invocation runs an entire script without spawning a process per operation.**

Two products emerged:

1. **exec** — the REPL, externalized as a CLI command any agent can use
2. **verify** — render, calc, lint as a lightweight add-on for agents that already have their own spreadsheet tools

The CLI comparison answered a question we hadn't quite asked: should the REPL be the external product? We'd built it for our own agent. This test showed exactly why it worked -- a single exec invocation runs an entire exploration script in one process, avoiding the per-operation overhead that killed the CLI approach.

So we externalized it. `witan xlsx exec` -- any coding agent can use it, not just ours. Changes are ephemeral by default and only persist with an explicit --save flag, so failed edits don't corrupt workbooks. The remaining CLI commands found their role as a lightweight verification add-on for agents that already use openpyxl or pandas.

Domain knowledge outlived every tool

— We changed tools four times in four months
— The financial domain knowledge improved results on **every one of them**

Structured as a composable prompt component we call the "Financial Expert Mindset":

— How to interpret margins, profitability, revenue cascades
— Model type recognition (DCF, LBO, three-statement)
— Communication conventions ($1.2M not $1,234,567.89)
— "Never calculate in your head what the spreadsheet can calculate for you"

It was the most reused component in the system.

We changed tools four times -- the domain knowledge survived all of them. How to interpret margins, what "profitability" actually asks for, that revenue changes cascade through COGS and working capital. These improved results regardless of what tool the agent was using.

We structured it as a composable prompt component. The "Financial Expert Mindset" could be attached to any tool approach. It became the most portable piece of the system.

Evaluation was half the work

We started with LLM-as-judge. We couldn't tell if a score change was the agent improving or the evaluator being flaky.

We replaced it with deterministic comparison wherever possible — programmatic checks for values, layout, and formatting. LLM grading only for genuinely subjective text answers.

568 commits. Nearly as complex as the agent itself.

The principle: match your evaluation method to your output type. If the comparison is objective -- are these values correct, is this layout right, do these formulas produce the right outputs -- use programmatic comparison. It's reproducible and you can trust score changes. Reserve LLM grading for cases where the output is genuinely subjective.

The scale surprised us. 568 commits, about 29,000 lines of TypeScript. Building reliable evaluation was as much engineering effort as building the agent. But every significant improvement in the project traced back to a benchmark result. Without evaluation you can trust, you're guessing.

Infrastructure bugs look like reasoning failures

What it looked like	What it actually was
Agent can't find data	One-character extraction bug (50% → 73%)
Agent uses wrong quoting	SKILL.md had backwards examples
Agent times out constantly	Per-cell recalculation query instead of batch
Agent retries endlessly	API returning empty results intermittently

When the agent seems confused, check the plumbing first.

This was a recurring theme throughout the project. Every time we thought the agent was confused or the model was failing, the actual problem was somewhere in the infrastructure.

The one-character extraction bug that looked like model confusion. Documentation with backwards examples that the agent followed faithfully. A performance bug that looked like the agent being slow. An intermittent API failure that looked like the agent retrying for no reason.

What generalizes

1. **Many small sequential tool calls = a bad scripting language.** Give the agent a real one.
2. **Build verification engines.** Formula calc, rendering, and linting close a feedback loop that compounds with model capability rather than being replaced by it.
3. **Interfaces are ephemeral, engines are durable.** The REPL works because coding is today's strongest model skill. That may not last.
4. **Structured reasoning > better tools.** "Define the end state before you act" caught more errors than any tool improvement.
5. **Domain knowledge outlasts tools.** Four backends in four months. The domain knowledge improved results on all of them.
6. **Match evaluation to output type. Check the plumbing first.** Use deterministic comparison for objective outputs, LLM grading for subjective ones. Agent "confusion" is usually infrastructure.

One: if your agent is making many small sequential tool calls that compose into a larger operation, you've reinvented a bad scripting language. Give it a real one. This applies anywhere -- data analysis, code generation, system administration.

Two: the verification engines -- formula calculation, rendering, linting -- closed a feedback loop that held across three frontier model releases. Each new model used the same loop more effectively. The engines compound with capability.

Three: the REPL is the best interface today because coding is where models are strongest. But capability profiles shift. If computer use catches up to coding, different interfaces might work better. The engines underneath are the durable investment.

Four: structured reasoning beats better tools. Making the agent define the end state before executing was more impactful than any tool we built.

Five: domain knowledge is the most portable asset. We went through four tool backends. The domain knowledge improved results on all of them and outlasted all of them.

Six: match your evaluation method to the output type. If the comparison is objective, use programmatic checks -- they're reproducible and you can trust score changes. Reserve LLM grading for genuinely subjective outputs. And when the agent seems confused, check the infrastructure before blaming the model.

50% → 92%. Four months.

We spent four months trying to constrain the agent into tighter interactions. It worked better when we gave it a programming environment and got out of the way.

Research log: github.com/witanlabs/research-log

We spent four months trying to make an LLM work with spreadsheets. We tried databases, specialized tools, multi-agent architectures, multiple representations. The breakthrough was giving up on constraining the agent and letting it program.

The REPL collapsed complexity, made the API trivially extensible, and eventually became the product itself. But the more durable insight was about the engines underneath -- formula calculation, rendering, linting. Those are what let each new model do better work on the same tasks.

Thank you.

WITAN